CAHIER DES CHARGES TECHNIQUE ET FONCTIONNEL

« LINCEUL AUDIT »

Code Projet : LINCEUL-2026
Classification : CONFIDENTIEL / HAUTE CRITICITÉ
Architecture Cible : Hybride Java 25 (Loom) / Python (Data Science) / ONNX
Conformité : AI Act (UE), RGPD, CMF L561-15, ISO/IEC 42001

---

1. Introduction et Contexte Stratégique

1.1. Préambule : Le Changement de Paradigme

Le secteur financier européen fait face à une double rupture : l'industrialisation des vecteurs d'attaque (fraude synthétique, réseaux de mules, attaques adverses sur IA) et un durcissement sans précédent du cadre réglementaire. Le projet « Linceul Audit » ne vise pas une simple mise à niveau incrémentale des systèmes de surveillance des transactions (TMS). Il ambitionne de **redéfinir l'épistémologie de la détection de fraude** : passer d'une approche probabiliste opaque (« Boîte Noire ») à une approche déterministe, explicable et juridiquement opposable (« Boîte Grise »).

L'objectif est de déployer une plateforme capable de traiter des flux massifs en temps réel avec une latence infra-milliseconde, tout en garantissant que chaque décision algorithmique soit auditable, équitable et conforme aux exigences de l'AI Act et de l'ACPR.

1.2. La Double Injonction : Performance vs Conformité

Le système doit résoudre l'équation suivante :

1. **Performance (Throughput & Latency) :** Absorber les pics de charge (C10k problem) sans dégradation de service, en exploitant la concurrence structurée de Java 21 (Virtual Threads).
2. **Rigueur Juridique (Compliance) :** Garantir que l'utilisation de threads virtuels et de modèles neuronaux (Deep Learning) ne compromet ni la traçabilité ni la stabilité du système (problématique du *Pinning* JNI).

---

2. Cadre Normatif Avancé et Conformité

2.1. AI Act et Gestion des Risques Fondamentaux

- **Classification (Annexe III) :** Bien que bénéficiant de l'exception pour la « détection de la fraude financière », le système doit prouver qu'il n'est pas utilisé pour le *scoring* de crédit (Exigence REG-01).
- **FRIA (Fundamental Rights Impact Assessment) :** Conformément à l'Art. 27 de l'AI Act, une étude d'impact sur les droits fondamentaux doit être intégrée. Le système

doit logger non seulement la décision, mais aussi la preuve de non-discrimination à l'instant T.

2.2. RGPD Art. 22 et l'Architecture HITL

L'interdiction de décision automatisée à effet juridique impose une architecture **Human-in-the-Loop (HITL)** stricte pour les alertes critiques. L'interface utilisateur n'est pas un accessoire, mais un **garant juridique**. Elle doit permettre à l'analyste de renverser la décision de la machine (Override) sur la base d'éléments explicables (SHAP Values).

2.3. LCB-FT et Traçabilité

L'obligation de résultat (Art. L561-15 CMF) impose une **journalisation WORM** (Write Once, Read Many). En cas d'enquête judiciaire 5 ans après les faits, la banque doit pouvoir « rejouer » la transaction avec le modèle exact de l'époque et obtenir le même résultat (Déterminisme absolu).

---

3. Architecture Technique : Haute Performance et Résilience

3.1. Stack Technologique Cœur

- **Orchestration & I/O :** Java 21 LTS avec **Project Loom (Virtual Threads)**.
- **Intelligence & Inférence :** Modèles entraînés en **Python**, exportés en **ONNX**, exécutés via **ONNX Runtime (Java API)**.
- **Données & Vecteurs :** PostgreSQL + **pgvector** (Recherche de similarité sémantique).

3.2. Gestion de la Concurrence et Mitigation du « Pinning »

Le problème critique identifié est le *Pinning* (épinglage) des threads virtuels lors des appels natifs (JNI) vers ONNX, bloquant les threads porteurs (Carrier Threads).

Stratégie de Mitigation (Pattern Bulkhead) :

1. **Front-End (I/O Bound) :** Utilisation massive de Virtual Threads pour la gestion HTTP, la validation JSON et les appels BDD non bloquants.
2. **Back-End Inférence (CPU Bound / JNI) :** Isolation stricte de l'inférence dans un `ThreadPoolExecutor` dédié (Platform Threads), dimensionné selon le nombre de cœurs physiques.
3. **Surveillance JFR :** Activation permanente de *JDK Flight Recorder* avec l'événement `jdk.VirtualThreadPinned` pour détecter toute régression en production.

3.3. Résilience et Mode Dégradé

- **Backpressure (Load Shedding) :** Implémentation d'un mécanisme de délestage. Si la file d'attente du pool d'inférence dépasse un seuil critique (ex: 500ms de lag), les nouvelles requêtes sont rejetées ou redirigées.
- **Fallback Heuristique :** En cas d'indisponibilité du moteur ONNX ou de latence excessive, le système bascule automatiquement sur un moteur de règles « en dur » (Rule-based engine) déterministe pour assurer la continuité de service.

---

4. Ingénierie des Données et DataOps

4.1. Feature Store et Lignage

Pour garantir la reproductibilité :

- **Feature Store :** Centralisation des définitions de variables. Chaque feature (ex: « montant moyen sur 7 jours ») doit avoir une définition unique et versionnée.
- **Data Lineage :** Capacité de tracer l'origine de chaque donnée (Source Core Banking → Transformation ETL → Tenseur d'entrée).

4.2. Qualité de Donnée

- **Imputation Stratégique :** Aucun modèle ne doit recevoir de valeurs `NaN` ou `Null`. Une stratégie d'imputation stricte (médiane, -1, ou rejet) doit être définie et documentée pour chaque champ.

---

5. Personas Détaillés : Ingénierie Sociotechnique

Persona 1 : « L'Analyste Investigateur »

Profil : Nora, 34 ans, Analyste Senior LCB-FT. Expert métier, sceptique vis-à-vis de l'IA « Magique ». Travaille sous pression temporelle.

- **Objectif :** Traiter 50+ alertes/jour sans erreur de jugement. Veut comprendre pourquoi une alerte est levée pour rédiger sa déclaration de soupçon.
- **Besoin Cognitif :** Réduction de la charge mentale. Interface de Data Lineage et hiérarchisation via les facteurs SHAP.
- **Exigence Système :** Latence d'affichage < 200ms. Explicabilité en langage naturel.

Persona 2 : « L'Architecte Data Scientist »

Profil : Dr. Ezra ARIS, 29 ans, PhD en Machine Learning. Expert Python/PyTorch.

- **Objectif :** Déployer ses modèles complexes (Auto-encodeurs, Random Forest, GNN) sans les réécrire en Java.
- **Besoin Technique :** Parité stricte entre Python et Java. Support total des opérateurs ONNX Runtime.
- **Exigence Système :** Pipeline CI/CD automatisé validant la fidélité à 10^{-6} près.

Persona 3 : « Le Gardien de la Conformité »

Profil : Axel, 52 ans, Juriste et DPO. Aversion totale au risque réglementaire.

- **Objectif :** Prouver à l'ACPR et à la CNIL que les modèles sont impartiaux.
- **Besoin Fonctionnel :** Bouton « Générer Rapport d'Audit » incluant la Matrice de Confusion et la version du modèle.
- **Exigence Système :** Logs immuables (WORM), Documentation vivante (Model Cards).

Persona 4 : « L'Ingénieur SRE »

Profil : Sophie, 40 ans, Expert JVM. Obsédée par la disponibilité (99.999%).

- **Objectif :** Éviter les crashs sous charge et le Pinning des Virtual Threads.
- **Besoin Technique :** Observabilité totale (Metrics Micrometer/Prometheus). Load Shedding automatique.
- **Exigence Système :** Isolation des ressources, Backpressure, Monitoring précis.

---

6. User Stories Exhaustives

Épic 1 : Ingestion et Prétraitement

- **US-01 (Intégrité) :** En tant que Système, je dois valider le schéma JSON de chaque transaction entrante et rejeter immédiatement les formats invalides.
- **US-02 (Enrichissement) :** En tant que Système, je dois récupérer les données historiques en moins de 10ms.
- **US-03 (Imputation) :** En tant que Data Scientist, les valeurs manquantes doivent être remplacées selon la stratégie définie.

Épic 2 : Inférence et Détection

- **US-04 (Performance) :** L'inférence doit s'exécuter dans un pool dédié sans bloquer les Virtual Threads.
- **US-05 (Fail-Safe) :** Bascule automatique sur mode « Règles simples » si latence > 500ms.
- **US-06 (Parité) :** Vérification automatique que le score Java = score Python.

Épic 3 : Interface Analyste et Décision

- **US-07 (Explicabilité) :** Affichage des 3 principaux facteurs SHAP en langage clair.
- **US-08 (Décision) :** Capacité à qualifier une alerte avec justification.
- **US-09 (Similitude) :** Liste de transactions passées similaires via pgvector.

Épic 4 : Audit et Conformité

- **US-10 (Traçabilité) :** Chaque décision génère un log cryptographique immuable.
- **US-11 (Reporting) :** Génération d'un pré-rapport PDF conforme à l'article L561-15 CMF.
- **US-12 (Droit d'accès) :** Extraction de l'historique pour répondre aux demandes RGPD.

Épic 5 : Sécurité et MLOps

- **US-13 (Sécurité Modèle) :** Le fichier `.onnx` est chiffré au repos.
- **US-14 (Anti-Adversaire) :** Détection des inputs atypiques signalant des attaques.
- **US-15 (Dérive) :** Alerte automatique si la distribution des scores dévie de > 5%.

---

7. Sécurité Avancée et Cyber-Résilience

7.1. Protection contre les Attaques Adversaires

L'IA est une surface d'attaque. Le système doit intégrer des défenses contre :

- **L'Evasion (Adversarial Examples) :** Perturbations mineures pour tromper le score.
  - *Mitigation :* Entraînement robuste et sanitization des entrées.
- **L'Empoisonnement (Data Poisoning) :** Injection de données frauduleuses dans l'entraînement.
  - *Mitigation :* Audit strict du Feature Store.
- **L'Inversion de Modèle :** Tentative de reconstruire les données d'entraînement.
  - *Mitigation :* Rate Limiting et arrondissement des scores.

7.2. Gestion des Secrets

Aucun mot de passe ou clé de chiffrement dans le code. Utilisation obligatoire d'un **KMS (Key Management Service)** ou de **HashiCorp Vault** pour l'injection des secrets au runtime.

---

8. Stratégie de Test et Qualité

Le plan de test doit être draconien pour valider le changement technologique :

1. **Tests de Charge :** Simulation de 5x la charge nominale pour vérifier le comportement des Virtual Threads et l'absence de fuite mémoire.
2. **Tests de Parité :** Validation croisée Python vs Java sur un dataset de référence (Golden Dataset).
3. **Audit de Code Statique :** Détection des blocs `synchronized` susceptibles de causer du Pinning.
4. **Backtesting :** Rejeu des 12 derniers mois de transactions sur le nouveau modèle avant bascule.

---

9. Conclusion et Validation

Ce cahier des charges définit un système **« State of the Art »**. Il ne se contente pas d'implémenter une technologie, il construit un **bouclier juridique et technique**. En intégrant l'auditabilité au cœur même de l'architecture, le projet « Linceul Audit » transforme la contrainte réglementaire en un atout de résilience opérationnelle.

Validation Requise

- [ ] Direction Technique (Architecture)
- [ ] Direction Data (Modélisation)
- [ ] Direction Conformité (DPO/Légal)
- [ ] Direction Métier (Fraude)

Résumé Exécutif

Linceul Audit est une plateforme de détection de fraude financière de **classe mondiale**, conçue pour l'ère de la conformité réglementaire strict (AI Act, RGPD, CMF). Elle combine :

- **Performance extrême :** Java 21 Virtual Threads + ONNX Runtime pour une latence < 1ms
- **Explicabilité juridique :** SHAP Values, HITL stricte, logs immuables WORM
- **Résilience absolue :** Bulkhead pattern, Load Shedding, Fallback heuristique
- **Sécurité renforcée :** Protection contre les attaques adverses, gestion sécurisée des secrets
- **Auditabilité permanente :** Data Lineage, Feature Store versionnée, Model Cards automatiques

Elle répond aux exigences de quatre personas clés dont les intérêts convergent vers une unique mission : **détecter la fraude avec certitude juridique**.

Document généré : Janvier 2026
Classification : CONFIDENTIEL / HAUTE CRITICITÉ
Auteurs : Architecture & Data Science Team